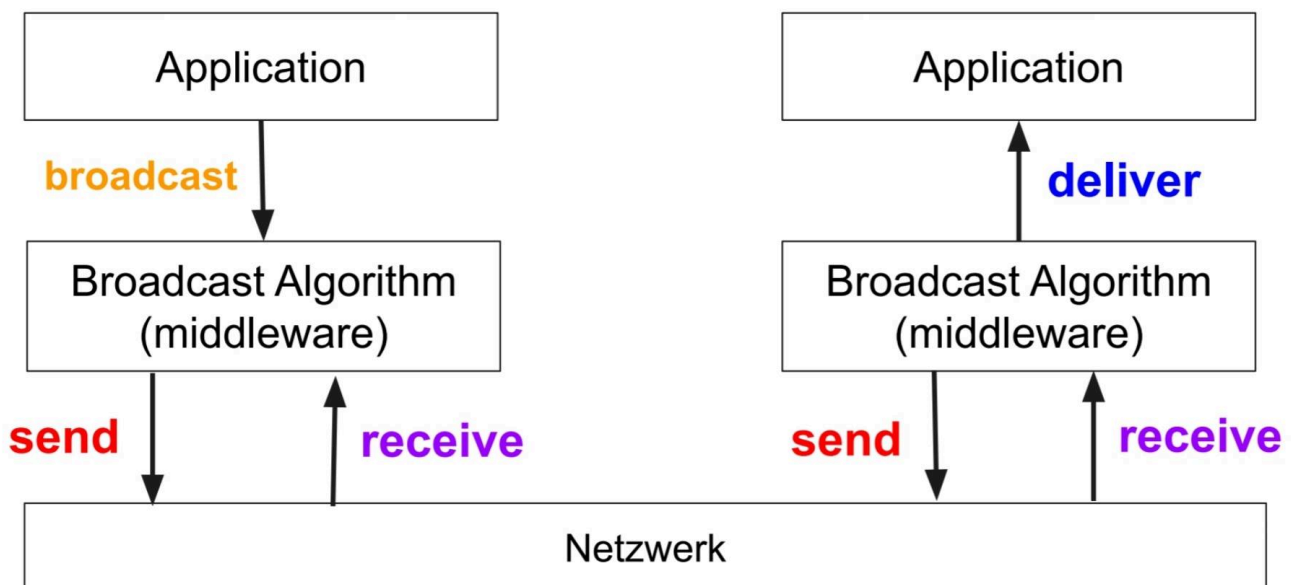


Broadcast Algorithmen

Gruppenkommunikation in der ein Knoten eine Nachricht an alle anderen Knoten einer Gruppe versendet. Dabei gibt es beliebig viele Knoten und das Ausfallen eines Knotens verhindert die Propagierung der Nachricht nicht.

- Peer to Peer Kommunikation
- kann best-effort oder reliable ablaufen
- oder asynchron/partiell synchron

Aufbau



Gossip Protokolle

Nachrichten werden an eine Untergruppe von Knoten versendet und dadurch schneller verbreitet

- Entstanden aus Eager Reliable Broadcast: Jeder Knoten startet bei Nachrichteneingang einen Re-Broadcast, damit nichts verloren geht in $O(n^2)$

Stärken

- Skalierbarkeit
 - $O(\log(n))$ für Erreichen aller Knoten
- Fehlertolerant
 - Besonders gut in partiell synchronen Systemen, weil gegen unbekannte Fehler gewappnet
 - Jeder Knoten sendet willkürlich an andere

- Knoten wird nächste Runde wieder erreicht
- Mehrere Wege auf einen Knoten zu kommen
- Robust
 - Keine Leader kein SPOF -> Kann auch munter Fehler übertragen
- Exponentiell konvergierend -> logarithmischer Abbau von unwissenden Knoten
- Dezentralisiert

Problem

In Gossip-Algorithmen könnte eine Nachricht theoretisch **unendlich weiter propagiert werden**, da jeder Knoten Nachrichten weiterverbreitet.

Die Lösung ist ein **Termination-Mechanismus**, der dafür sorgt, dass Knoten irgendwann aufhören, eine Nachricht zu verbreiten.

Quelle: Vorlesung *Broadcast-Algorithmen und kausale Ordnung von Nachrichten*.

Grundidee

Knoten besitzen Zustände (SIR-Modell):

- **Susceptible** -- hat die Nachricht noch nicht erhalten
- **Infective** -- hat die Nachricht und verbreitet sie
- **Removed** -- verbreitet die Nachricht nicht mehr

Sobald ein Knoten **Removed** wird, sendet er das Update nicht weiter.

Methoden zur Terminierung

1. Counter-basierter Removal

Nach **k erfolglosen Weitergabeversuchen** wird der Knoten entfernt.

```
if failed_updates >= k:
    state = Removed
```

Dadurch endet die Weiterverbreitung automatisch.

2. Random / Coin Removal

Nach einem erfolglosen Versuch wird zufällig entschieden:

```
probability = 1/k
if random_event:
    state = Removed
```

Die Anzahl der aktiven Sender sinkt mit jeder Runde.

3. Feedback Removal

Der Removal-Algorithmus wird ausgeführt, wenn der kontaktierte Knoten bereits infiziert ist.

Das signalisiert, dass die Nachricht bereits verbreitet wurde.

4. Blind Removal

Der Removal-Test wird **in jedem Zyklus** ausgeführt, unabhängig davon, ob ein Knoten bereits infiziert ist.

Warum das funktioniert

Mit jeder Runde:

- Anzahl **Infective** Knoten sinkt
- Anzahl **Removed** Knoten steigt

Die Propagation stoppt nach wenigen Runden.

Klausurantwort (Kurzform)

In Gossip-Algorithmen verhindert ein **Removal-Mechanismus** die unendliche Weiterverbreitung von Nachrichten. Knoten wechseln nach einer bestimmten Anzahl von Versuchen oder probabilistisch in den Zustand **Removed** und verbreiten die Nachricht danach nicht mehr.

Kausale Ordnung

Sei $\prec$ eine strikte totale Ordnung von Ergebnissen einer Menge E .

Falls es eine Happens-before Relation gibt, so dass stets aus $(a \rightarrow b) \Rightarrow (a \prec b)$ folgt, nennt man $\prec$ auch eine kausale Ordnung der Ergebnismenge E .

Daraus folgt, dass Happens-before keine zwingende Kausalität impliziert, aber Kausalität erfordert Happens-before in informationstechnischen Systemen.

Happens-before Relation

Ergebnis a findet vor Ergebnis b statt, wenn:

- a und b auf dem gleichen Knoten stattfinden und a bzgl. der physikalischen Ausführungsreihenfolge vor b ausgeführt wurde
- a das Senden und b das Empfangen derselben Nachricht m ist
- es ein Ergebnis c gibt, sodass $a \rightarrow c$ und $c \rightarrow b$ gilt ([Transitivität](#))

Immer nur eine partielle Ordnung:

- Wenn a und b nebenläufig sind: $a \parallel b$

Weiteres:

- a muss nicht zwingend zeitlich früher sein, sondern es **könnte** ein kausaler Zusammenhang im lokalen Bereich bestehen
- Es ist eine strikte partielle Ordnung, weswegen Abbildungen von potentiellen Kausalität in Verteilten Systemen möglich ist

Strikte partielle Ordnung

Relation R ist strikt partiell geordnet, wenn sie irreflexiv und transitiv ist

- Ergebnisse sind entweder $a \rightarrow b$ oder $b \rightarrow a$ oder $a \parallel b$

Logische Uhren

Logische Uhren(T) zählen Ereignisse anstatt von Zeit im Format $(a \rightarrow b) \Rightarrow (T(a) < (b))$

- Physikalische Uhren zählen Sekunden
- Logische Uhren zählen Ereignisse

Lamport-Uhren

Eine Uhr, die anstelle der Zeit, Ereignisse zählt

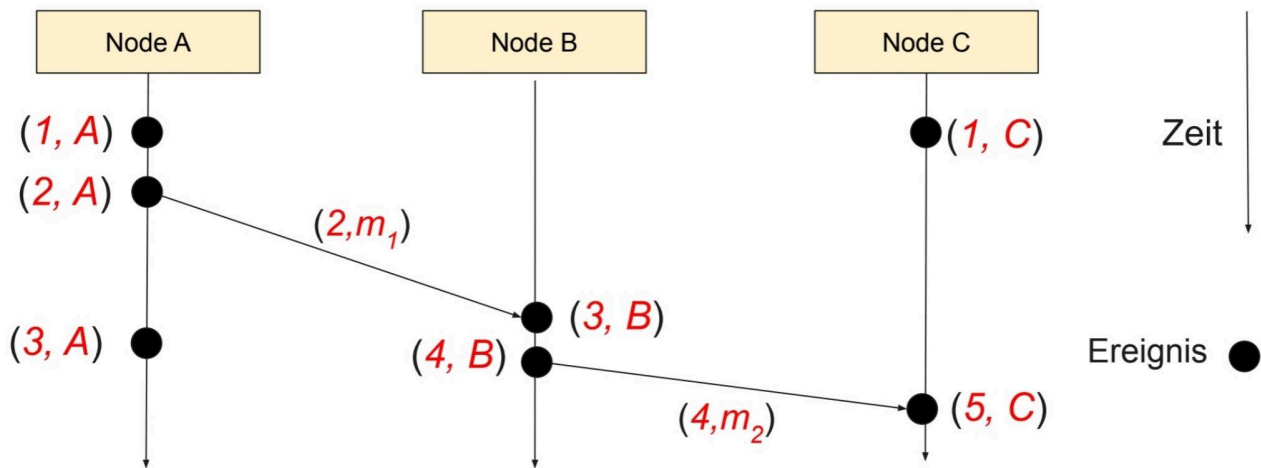
Algorithmus

- Jeder Knoten hat seinen eigenen Zähler t , der bei jedem lokalen Ereignis um 1 erhöht wird
- Beim Senden einer Nachricht wird dieser zusammen mit der Nachricht (t, m) verschickt
- Beim Empfangen von (t', m) wird das Maximum zwischen eigenen Zähler t und t' bestimmt und um 1 inkrementiert. Dies ergibt den neuen Wert des lokalen Zählers

Wenn $L(x)$ der durch diesen Algorithmus bestimmten Lamport-Uhrwert für das Ergebnis x ist, dann gelten die folgenden Aussagen über eine Lamport-Uhr

- Falls $(a \rightarrow b) \Rightarrow (L(a) < L(b))$
- Aber, aus $(L(a) < L(b))$ folgt nicht $(a \rightarrow b)$
- Außerdem ist es möglich, dass $L(a) = L(b)$ für $a \neq b$

Beispiel



- Sei $N(e)$ der Name des Knotens, auf dem Ereignis e stattgefunden hat. Das Paar $(L(e), N(e))$ bestimmt dann eindeutig das Ergebnis e .
- Man kann dadurch die totale Ordnung $\prec$ auf der Ereignismenge unter der Nutzung der Lamport-Zeitstempel der lexikographischen Ordnung der Knotennamen wie folgt definieren. #

$$(a \prec b) \Leftrightarrow (L(a) < L(b) \vee (L(a) = L(b) \wedge N(a) < N(b)))$$

- Die Resultierende Ordnung ist dann kausal
- Nebenläufigkeit wird willkürlich basierend auf den Knoten eingereiht

Vektoruhren

Selbe Logik wie Lamport-Uhr, nur mit dediziertem Counter pro Knoten

- Warum?
 - Mit Lamport-Zeitstempeln können wir nicht zwischen $(a \rightarrow b)$ oder $(a || b)$ unterscheiden
 - ABER, exakte Reihenfolge von Nebenläufigen Ereignissen auch nicht entscheidbar

Ordnung

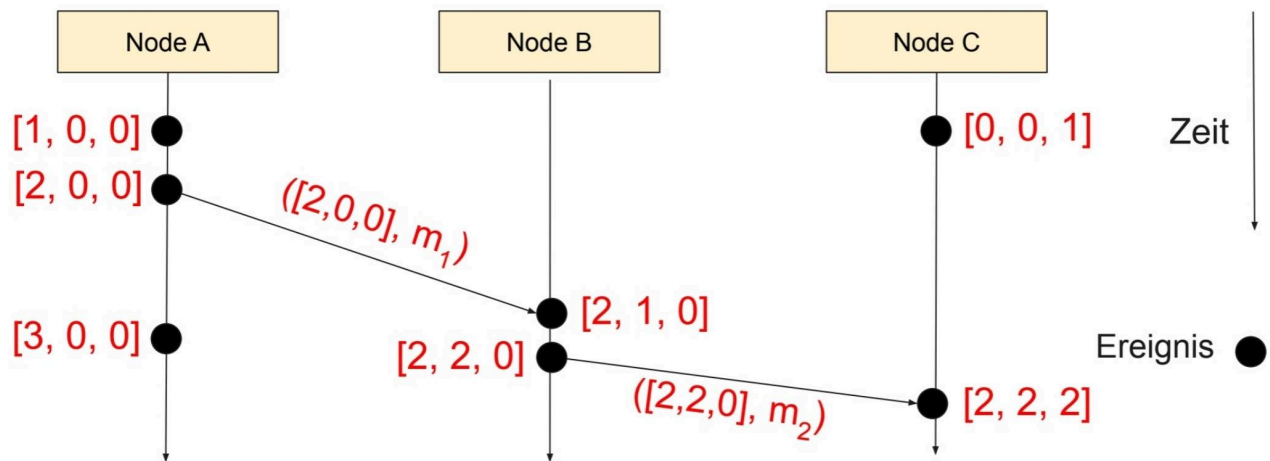
Mit Vektor-Uhren können folgende Ordnungen V definiert werden

- $T = T'$ falls $T[i] = T'[i]$ für alle $i \in \{1, \dots, n\}$
- $T \leq T'$ falls $T[i] \leq T'[i]$ für alle $i \in \{1, \dots, n\}$
- $T < T'$ falls $T \leq T'$ und $T \neq T'$
- $T || T'$ falls $T \not\leq T'$ und $T \neq T'$

$$V(a) \leq V(b) \text{ bei } (\{a\} \cup \{e | e \rightarrow a\} \subseteq \{b\} \cup \{e | e \rightarrow b\})$$

Die resultierende Ordnung V hat somit dieselben Eigenschaften wie Happens-before und ist genau eine Implementierung dieser

Beispiel



Vektor-Zeitstempel eines Ergebnisses e beschreibt eine Menge von Ereignissen: zum einen e selbst, sowie seine kausalen Abhängigkeiten $\{e\} \cup \{a | a \rightarrow e\}$

So ist $[2, 2, 0]$ z.B. oben die ersten zwei Ereignisse von A sowie die ersten 2 von B und keine von C.

Anwendung

- Erkennung von Schreibkonflikten
- Kausale Auslieferung
- Debugging in Verteilten Systemen -> dem Nachrichtenkanal zuweisbar

Hybrid Logical Clock (HLC)

Ein Zeitspempel, der kausal konsistent ist, sich aber so nah wie möglich an der echten Wanduhr bewegt

Kombiniert die Genauigkeit der physischen Zeit mit Verlässlichkeit logischer Uhren

Zeitstempel

Tripel (pt, l, i)

- $pt \rightarrow$ physical time: Das maximale Wissen über die physische Zeit (Unix-Nanosekunden)
- $l \rightarrow$ logical counter: Ein Zähler für Ereignisse, die "gleichzeitig" bzw. innerhalb derselben Zeiteinheit stattfinden
- $i \rightarrow$ node id: Eindeutige Kennung des Knotens (als Tie-Breaker)

Abläufe

- Lokales Ereignis auf einen Knoten
 - Knoten überprüft seine aktuelle Systemzeit (wall_clock)
 - Echte Uhr läuft vor -> Wenn $wall_clock > pt$, wird das neue pt auf $wall_clock$ gesetzt und der Zähler $l = 0$
 - Uhr ist hinterher -> Wenn $wall_clock \leq pt$, behält der Knoten das alte pt bei und erhöht den Zähler um $l = l + 1$

- Senden einer Nachricht
 - Der Knoten aktualisiert seinen lokalen HLC-Zeitstempel (wie oben) und hängt diesen (pt, l, i) an die zu sendende Nachricht an
 - Empfangen einer Nachricht
 - Knoten B erhält eine Nachricht mit $T_{msg} = (m.pt, m.l, m.i)$. Er muss nun seinen eigenen Zeitstempel T_{local} anpassen
1. Maximum finden: $pt_{neu} = \max(T_{local}.pt, m.pt, wall_clock)$
 2. Logischen Zähler anpassen
 1. Wenn $pt_{neu} = T_{local}.pt \wedge pt_{neu} = m.pt: l = \max(T_{local}.l, m.l)$
 2. Wenn $pt_{neu} = T_{local}.pt: l = T_{local}.l + 1$
 3. Wenn $pt_{neu} = m.pt: l = m.i + 1$
 4. Wenn $pt_{neu} = wall_clock: l = 0$

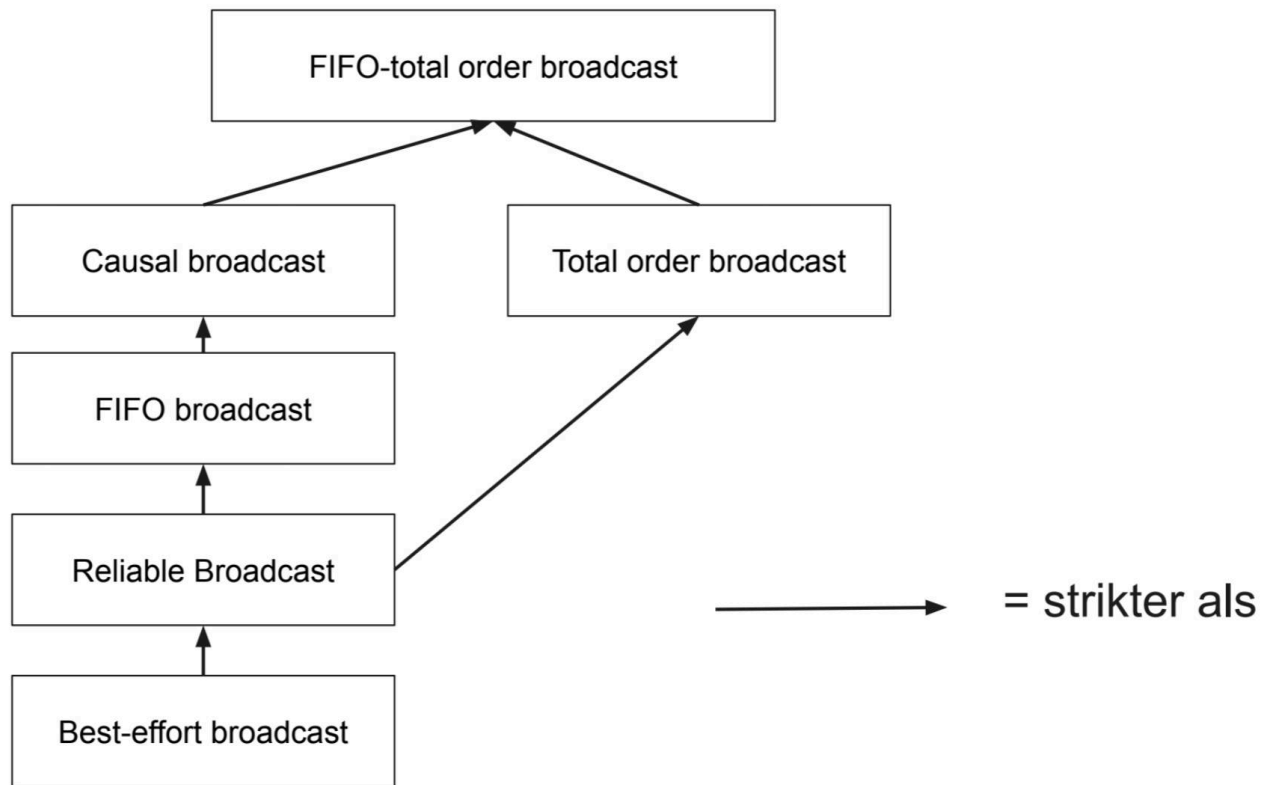
Eigenschaften

- Umgang mit "Clock Skew":
 - Knoten übernimmt immer den höheren pt-Wert bei Übertragungen
 - Kausalität somit auch bei Drift gewährt
- Dank Wallclock driftet die Uhr nicht beliebig sondern mit Tendenz zu Wallclock
- Monotonie: Uhren können nicht rückwärts gestellt werden

Anwendung

- Systeme mit Kausalität und totaler Ordnung für Last Write Wins
- Transaktionsmanagement bei verteilten DBs z.B. MongoDB
- Ersatz für Atomuhren
- Feste Bitlänge -> skaliert besser -> Chat, Eventsysteme usw.

Broadcast Protokolle und Ordnungsrelationen



FIFO Broadcast

Sendungsordnung EINES Knoten

- Nachrichten vom selben Knoten müssen in derselben Reihenfolge geliefert werden
- Nachrichten von verschiedenen Knoten können in beliebiger Reihenfolge geliefert werden
- Verletzt Kausalität
- Festhalten der Nachrichtenreihenfolge mit sendSeq Nummer

Kausaler Broadcast

Sendungsordnung Knotenübergreifend

- Nachrichten werden in Kausaler Reihenfolge geliefert
- Nebenläufige in beliebiger Reihenfolge
- Analog zur Vektoruhr wird das komplette Array versendet

Total Order Broadcast

Alle Replika verteilen die Nachricht in derselben Reihenfolge

- Auch atomic Broadcast
- Alle Knoten müssen die Nachrichten in derselben Reihenfolge versenden, auch wenn es zu Verzögerungen dadurch kommt
- Betrifft auch Auslieferungen auf den sendenden Knoten selbst
Umsetzung entweder durch Single Leader oder Lamport-Uhr

FIFO-Total Order Broadcast

| Kausales FIFO über Gesamtcluster